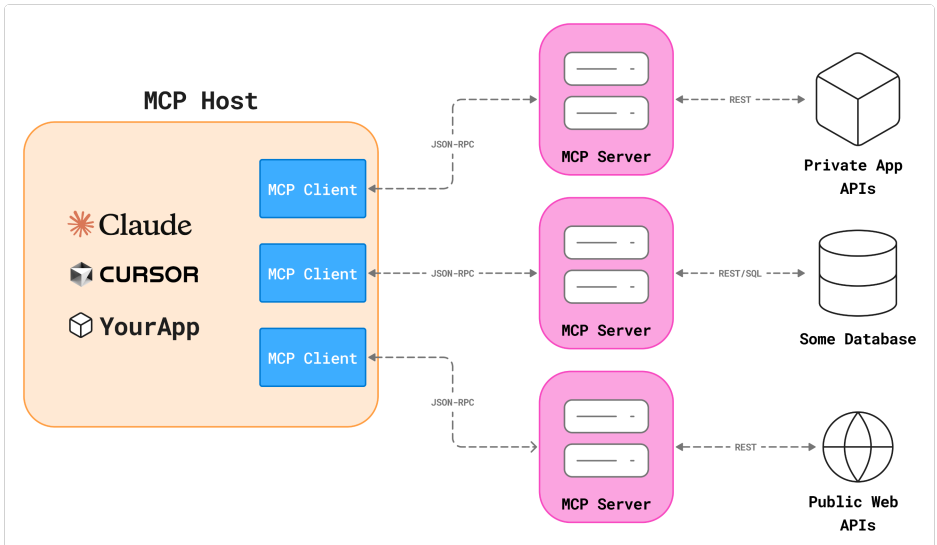


BUILDING YOUR OWN MCP SERVER FROM SCRATCH



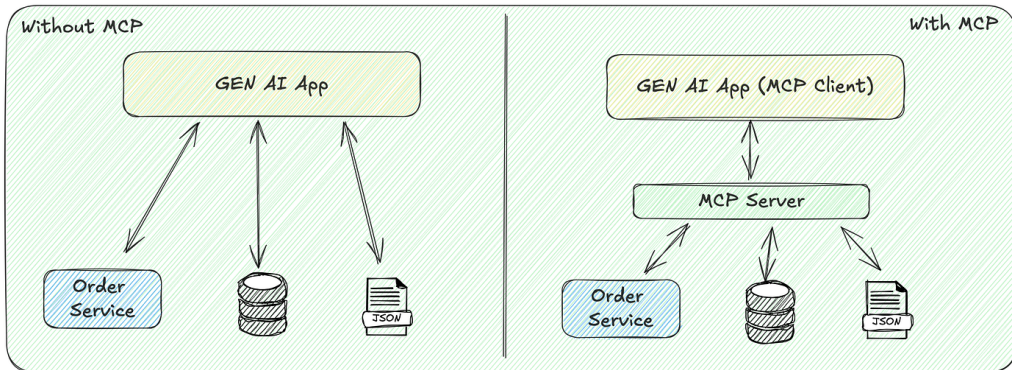
! CONTEXT

The Problem

🔒 AI models are geniuses locked in a windowless room.

They were trained on the internet up to a date, then frozen. They know everything in general and nothing about you.

- ❌ "What tasks are due today?" → **doesn't know**
- ❌ "Check our Postgres staging DB" → **can't reach it**
- ❌ "Create the Jira ticket" → **can't act**



× LIMITATIONS

Why Copy-Paste Isn't The Answer

"Just paste the data in!" - okay, but:



Manual Labor: You do it manually, every single time.



Size Limits: Context windows have limits. You cannot paste a 4GB database.

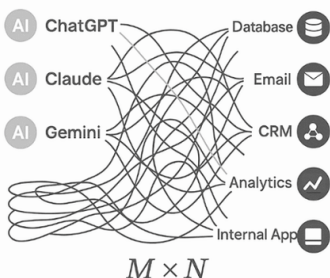


Stale Data: It is a snapshot, stale the moment you send it.

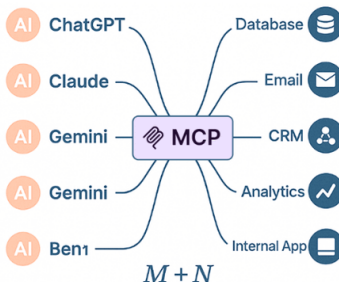


One-Way Street: Pasting lets AI read. It still cannot write, trigger, or update.

Without MCP



With MCP



The Real Pain: $M \times N$

Before MCP, every connection was hand-built.

5 AI apps $\times$ 20 tools = **100 custom integrations**

- Claude built its own Slack connector.
- Cursor built a different Slack connector.
- Slack built adapters for each AI vendor.

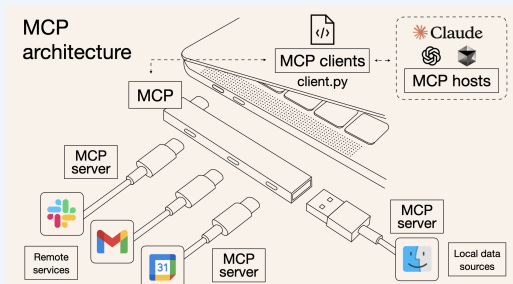
Same logic. Rebuilt. Over and over. Each one with its own auth, its own bugs, its own maintenance bill.



It is the pre-USB world of proprietary dongles.

What Is MCP?

 Model Context Protocol = **USB-C for AI.**



An open standard built on JSON-RPC 2.0, released by Anthropic in Nov 2024 and now adopted across the industry (including OpenAI, Google, Cursor, and VS Code).

$M \times N \rightarrow M + N$

Build your server once. Every compatible AI client can use it. Zero rewrites. Apache 2.0 licensed. Free.

MCP & What You Already Know

Feature	REST API	Function Calling	MCP
Built for	Humans/apps	One model	Any AI client
Discovery	Read the docs	Hardcoded list	Auto-discovered
Portable	-	Vendor-locked	Works everywhere
Live conn.	Request/resp.	Per-call	Persistent session

→ MCP doesn't replace your API. It wraps it so AI can use it.

✓ **Function calling** = "here's what you can do."

✓ **MCP** = "here's how to plug anything in."



ARCHITECTURE

The 3 Pieces



Host: the app you use (Claude Desktop, Cursor, your agent).



Client: lives inside the host, one dedicated connection per server.



Server: what you build. Exposes your data & actions.

You → Host → Client ↔ Your Server → Your DB/API



TRANSPORTS

>_ stdio: local, runs as a subprocess on your machine.



Streamable HTTP: remote, lives at a URL, multi-user.

Start with stdio. Ship to HTTP later.

What Actually Happens (The Handshake)

You never write this - but knowing it kills the magic-black-box feeling:

- 1 Host launches your server and opens a connection.
- 2 **Initialize:** both sides swap protocol versions & capabilities.
- 3 **tools/list:** client asks: "what can you do?"
- 4 Server replies with names + schemas + descriptions.
- 5 Model sees the menu and picks a tool.
- 6 **tools/call** → your Python function runs.
- 7 Result goes back → model writes the answer.



The model never touches your database. It only asks your server to.

What a Server Can Expose

TOOLS

Actions the AI calls → `create_task()`

Model-controlled. Can have side effects.

RESOURCES

Data the AI loads → `files, DB rows, docs`

App-controlled. Read-only, no side effects.

PROMPTS

Templates the user triggers → `/weekly-review`

User-controlled. Shows up as a slash command or button.

Memory hook: Tools = verbs. Resources = nouns. Prompts = shortcuts.

→ 90% of servers start with Tools only.

Real Servers In The Wild

 **GitHub** → read repos, open PRs, review code

 **Postgres** → query your DB in plain English

 **Filesystem** → read/write local folders

 **Slack** → search channels, post messages

 **Stripe** → pull payments & customers

 **Puppeteer** → drive a real browser

But the best ones are boring and internal: your team's deploy script, your legacy CRM, the ops runbook nobody's automated.



2,000+ public servers exist. The valuable one is the one only you can build.

Our Example

We are building: **A Task Tracker MCP Server** ✓

By the end, you will type in Claude:

"Add a task to finish the pitch deck by Friday, then show me everything pending."

...and Claude will actually do it - against your list.

STACK
Python + FastMCP

TIME
~10 minutes



Why this example? It has reads, writes, and state - the same shape as 90% of real servers.

>_ CODE

Step 0: Setup

Prereqs: Python 3.10+ (3.12+ recommended)

```
uv init task-server
cd task-server
uv add "fastmcp==3.2.4" # pin it - the API moves fast
```

(pip works too)

Create one file: **server.py**

⚡ WHY FASTMCP?

FastMCP powers ~70% of MCP servers. It handles schemas, validation, and transport so you write plain Python.

`</>` CODE

Step 1: Create The Server

```
from fastmcp import FastMCP

mcp = FastMCP("Task Tracker")

tasks = []  # our "database" (a list, for now)

if __name__ == "__main__":
    mcp.run()
```

 **FastMCP("Task Tracker")** names your server - this is what shows up in Claude's UI.

 **mcp.run()** starts listening on stdio.



You now have a valid MCP server. It just does nothing yet!

Step 2: Your First Tool

```
@mcp.tool
def add_task(title: str, due: str = "someday") -> str:
    """Add a new task to the user's task list.
    Use when the user wants to remember or schedule something."""
    tasks.append({"title": title, "due": due, "done": False})
    return f"Added: {title} (due {due})"
```

Three things do all the work:

1. `@mcp.tool` → registers it for discovery
2. **Type hints** → auto-generate JSON schema and validate inputs
3. **Docstring** → the model's only instruction manual



Your docstring is prompt engineering. Say what it does AND when to use it. Write it for a new intern.

Step 3: Read & Update Tools

```
@mcp.tool
def list_tasks(only_pending: bool = True) -> list:
    """List the user's tasks. Set only_pending=False to include
    completed."""
    if only_pending:
        return [t for t in tasks if not t["done"]]
    return tasks

@mcp.tool
def complete_task(title: str) -> str:
    """Mark a task as done, matched by its title."""
    for t in tasks:
        if t["title"].lower() == title.lower():
            t["done"] = True
            return f"✅ Completed: {title}"
    return f"No task found named '{title}'. Try list_tasks first."
```



Notice that last line - the error message teaches the model what to do next.

Step 4: Add a Resource

 Tools = do things.  Resources = read things.

```
@mcp.resource("tasks://summary")
def task_summary() -> str:
    """A quick overview of the task list."""
    done = sum(t["done"] for t in tasks)
    return f"{len(tasks)} tasks total, {done} done, {len(tasks)-done} pending."
```

`tasks://summary` is a URI - an address the host can pull context from without invoking an action.

 **Think: attaching a file vs. clicking a button.**

Step 5: Test Before You Ship

```
fastmcp dev server.py
```

Opens the MCP Inspector in your browser:

- ✓ See every tool + its generated schema
- ✓ Fire test calls manually
- ✓ Watch raw JSON-RPC messages
- ✓ Catch errors before the model does



If it's broken here, it's broken everywhere. Debug here first - always.

Step 6: Connect It To Claude

```
{
  "mcpServers": {
    "task-tracker": {
      "command": "uv",
      "args": ["--directory", "/full/path/to/task-server",
               "run", "server.py"]
    }
  }
}
```



~/Library/Application

Support/Claude/claude_desktop_config.json



%APPDATA%\Claude\claude_desktop_config.json

Fully restart Claude (quit, don't just close). Look for the tools icon 

Same server also plugs into Cursor, VS Code, and any MCP client - different config file, identical server.

 RESULTS

It's Alive 🥂

 **You:**

"Add 'finish pitch deck' due Friday."

 Claude calls `add_task` → "Added: finish pitch deck (due Friday)"

 **You:**

"What's still pending?"

 Claude calls `list_tasks` → your real list



You didn't train a model.

You didn't write a line of protocol code.

You wrote 4 Python functions.

When It Doesn't Work

Server won't appear?

→ Check logs: `~/Library/Logs/Claude/` (macOS) · `%APPDATA%\Claude\logs` (Windows)

"Server disconnected"?

→ Almost always a relative path or a missing dependency. Use absolute paths.

Model ignores your tool?

→ Your docstring is too vague. Add when to use it.

Model calls it with garbage arguments?

→ Your type hints are too loose. Narrow them (Literal, enums, tighter types).

6 Beginner Gotchas

1. **Never print() in a stdio server** - stdout is the protocol channel. Log to stderr.
2. **Absolute paths in config.** Relative paths fail silently.
3. **Return errors as text, don't crash.** "No task found" > a traceback.
4. **20 tools is too many.** Models get confused. Ship 3-5 sharp ones.
5. **Restart the host after config changes.** Every time.
6. **Pin your version.** A minor bump can change the constructor.

Security: Read This Twice

Your server runs with your permissions. Treat it like a public endpoint.

-  **Never hardcode secrets** - use env vars
-  **Least privilege** - a read-only DB tool should use a read-only DB user
-  **Assume tool inputs are untrusted** - the model can be steered by content it reads
-  **Destructive tools need guardrails** - confirmations, dry-run modes, soft deletes
-  **Log every call** - you'll want the audit trail

"The AI did it" is not an incident report.

When NOT To Build One

✗ SKIP MCP IF:

- A public server already exists → go check the registry first
- You need a one-off answer → just paste the data
- The task is pure computation with no external state
- It's a deterministic pipeline → that's a cron job, not an agent



MCP shines when: the data is yours, it changes, and the task is different every time.

[↑ NEXT STEPS](#)

Level Up

Once it works locally:

- Swap the list for SQLite / Postgres
- Switch to Streamable HTTP to deploy remotely
- Add OAuth for real multi-user auth
- Add Prompts for reusable workflows
- Add OpenTelemetry for production tracing
- Publish to the MCP registry

Same 4 functions. Production scale.